

HỒI QUY TUYẾN TÍNH

Linear Regression — Mô hình, Tối ưu hóa & Đánh giá

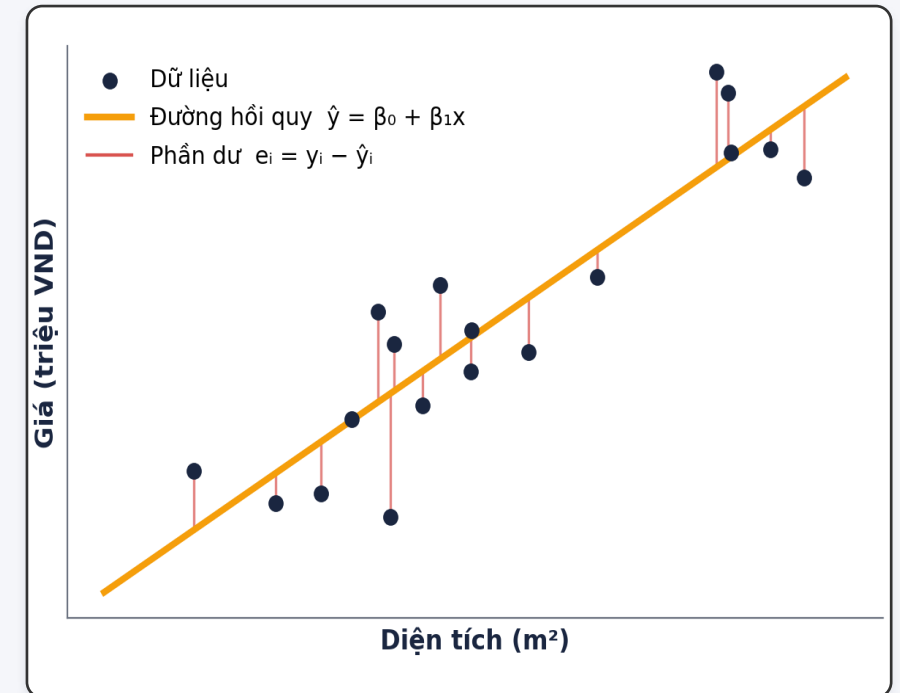
TS. Cao Tiến Dũng



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

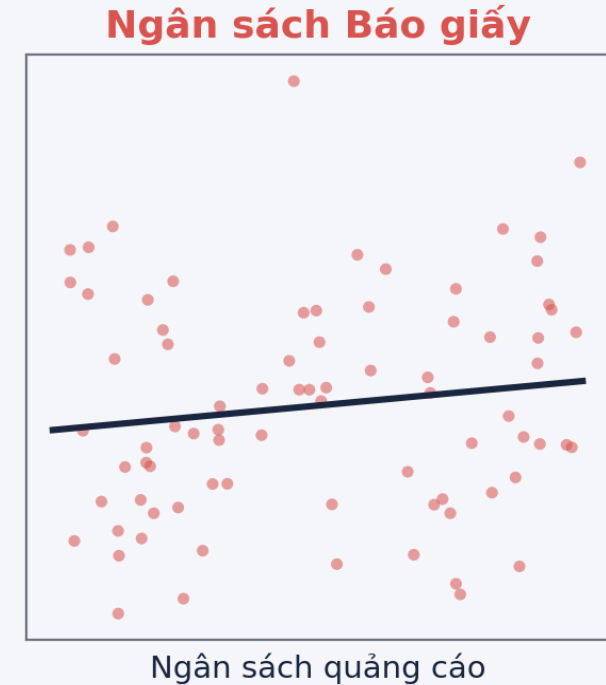
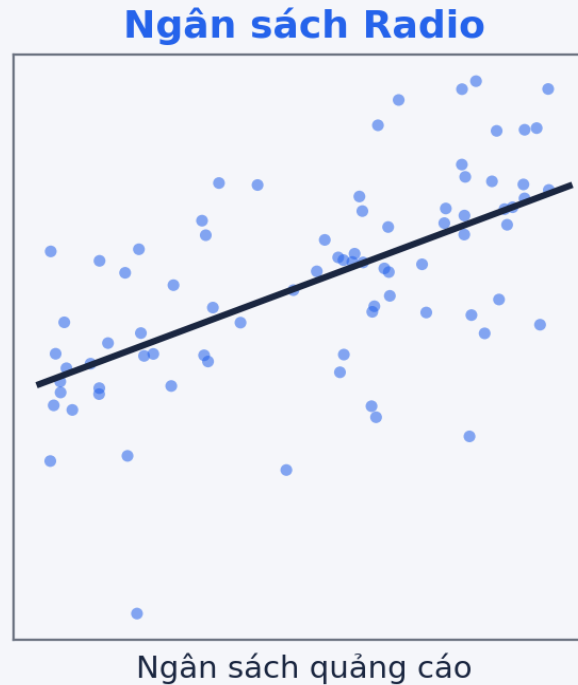
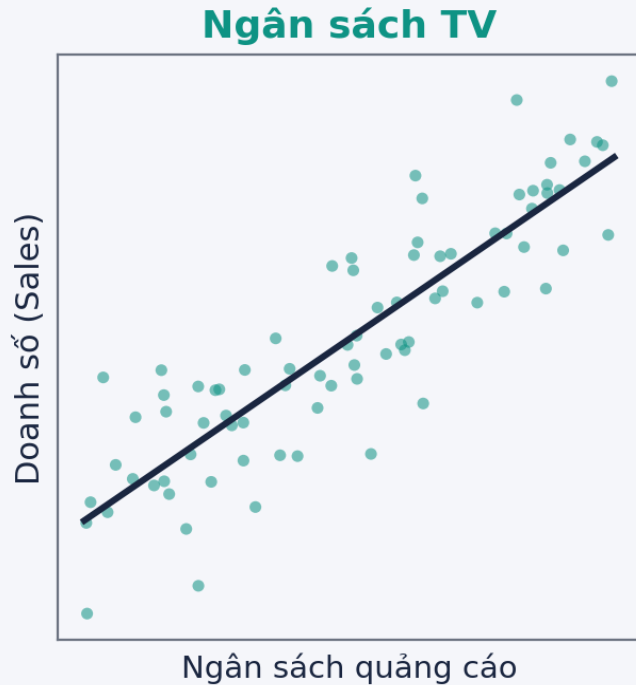
Hồi quy (Regression) là gì?

- Là quá trình xác định MỨC ĐỘ liên hệ giữa một biến phụ thuộc Y và một hoặc nhiều biến độc lập X.
- **Hồi quy tuyến tính:** giả định Y phụ thuộc TUYẾN TÍNH vào X (bài toán thực tế hiếm khi tuyến tính hoàn hảo).
- Ví dụ thực tế:
 - Dự đoán giá nhà (Y) từ diện tích, số phòng, vị trí (X).
 - Ước lượng lương (Y) từ số năm kinh nghiệm (X).
 - Dự báo doanh số (Y) từ ngân sách quảng cáo (X).



Y phụ thuộc X — tìm đường thẳng mô tả quan hệ

Hồi quy tuyến tính vs Dữ liệu



- **Dữ liệu quảng cáo:** TV có xu hướng tuyến tính **MẠNH** với doanh số; Radio **TRUNG BÌNH**; Báo giấy **YẾU**.
- **Nhận định then chốt:** không phải đặc trưng nào cũng hữu ích như nhau cho dự đoán — cần chọn lọc đặc trưng.

01

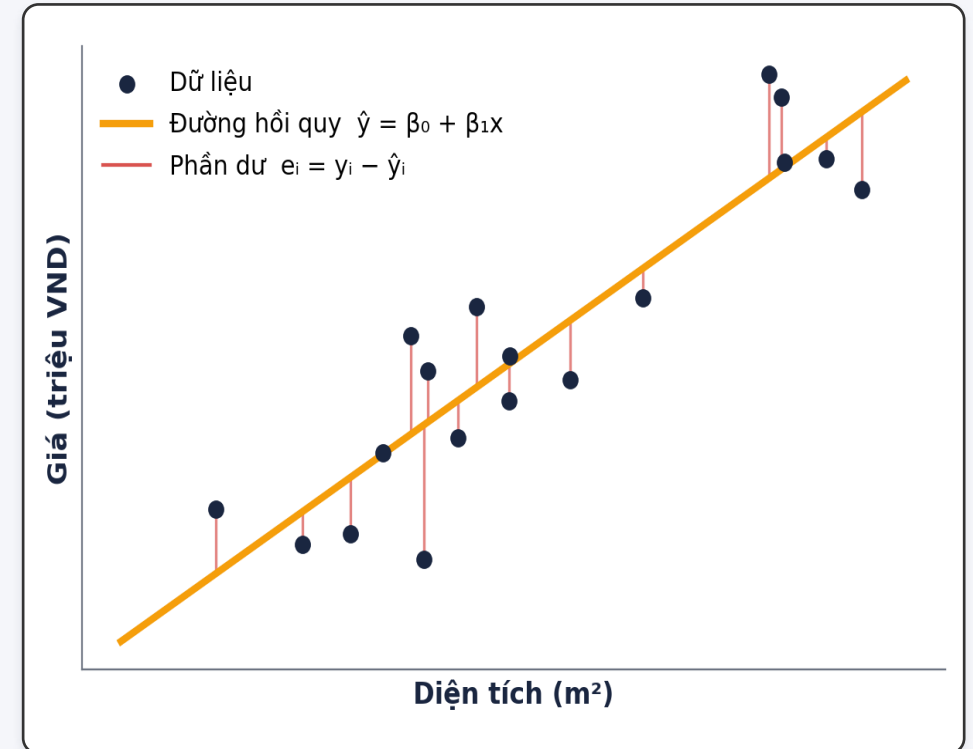
Mô hình & Hàm mất mát

Model & Loss Function

Mô hình tuyến tính đơn biến

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

- β_0, β_1 : hệ số chặn (intercept) và độ dốc (slope) — gọi chung là tham số.
- ε : sai số (error term).
- Hồi quy = tìm β_0, β_1 từ dữ liệu (X, Y) , rồi dùng để dự đoán Y tương lai.
- **Ví dụ:** Giá = $50 + 0.8 \times$ Diện tích. Nếu Diện tích = $100 \text{ m}^2 \rightarrow$ Giá = 130 (triệu VND).



Bình phương tối thiểu & RSS

- Dự đoán: $\hat{y}_i = \beta_0 + \beta_1 x_i$. **Phần dư (residual):** $e_i = y_i - \hat{y}_i$.
- Tổng bình phương phần dư (RSS):

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2$$

- **Phương pháp bình phương tối thiểu:** chọn β_0, β_1 để TỐI THIỂU RSS (hay RSS/2 — gọi là hàm chi phí).
- **Ví dụ:** $y=[3,5,7]$, $\hat{y}=[2.8, 5.1, 7.3] \rightarrow RSS = 0.2^2 + 0.1^2 + 0.3^2 = 0.14$.

Hàm mất mát: MSE (Sai số toàn phương trung bình)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **MSE = RSS / n** — trung bình bình phương sai số; là hàm mất mát chuẩn của hồi quy.
- **Vì sao bình phương?** phạt nặng sai số lớn, khả vi (dễ lấy đạo hàm để tối ưu), và có nghiệm dạng đóng.

MAE — sai số tuyệt đối

- Trung bình |sai số|; ít nhạy với điểm ngoại lai hơn MSE.

RMSE — căn của MSE

- Cùng đơn vị với Y → dễ diễn giải. (Chi tiết ở mục Đánh giá.)

Nghiệm dạng đóng cho hồi quy đơn

- Tối thiểu hàm chi phí: cho đạo hàm riêng theo β_0 và β_1 bằng 0.
- Giải hệ phương trình, thu được nghiệm trực tiếp:

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

- **Ý nghĩa:** độ dốc β_1 = hiệp phương sai(X,Y) / phương sai(X); đường hồi quy luôn đi qua điểm trung bình $(\bar{x}, \bar{y})$.

02

Hồi quy bội & Normal Equation

Multiple Regression & Normal Equation

Hồi quy tuyến tính bội (Multiple LR)

- Khi có NHIỀU đặc trưng (p đặc trưng, m mẫu), mô hình mở rộng thành:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

Dạng ma trận:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

- $\mathbf{y}$ ($m \times 1$): véc-tơ giá trị quan sát.
- $\mathbf{X}$ ($m \times (p+1)$): ma trận đặc trưng (cột đầu toàn số 1 cho hệ số chặn).
- $\boldsymbol{\beta}$ ($(p+1) \times 1$): véc-tơ tham số cần tìm.

Normal Equation (Phương trình chuẩn)

- Nghiệm bình phương tối thiểu cho hồi quy bội có công thức ĐÓNG (không cần lặp):

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

- **Trực tiếp** cho ra β tối ưu trong một phép tính — không cần chọn tốc độ học hay lặp.
- **Điều kiện:** $\mathbf{X}^\top \mathbf{X}$ phải KHẢ NGHỊCH (không có đa cộng tuyến hoàn toàn).
- **Chi phí:** nghịch đảo ma trận tốn $O(p^3)$ → chậm khi số đặc trưng p rất lớn.

Normal Equation vs Gradient Descent

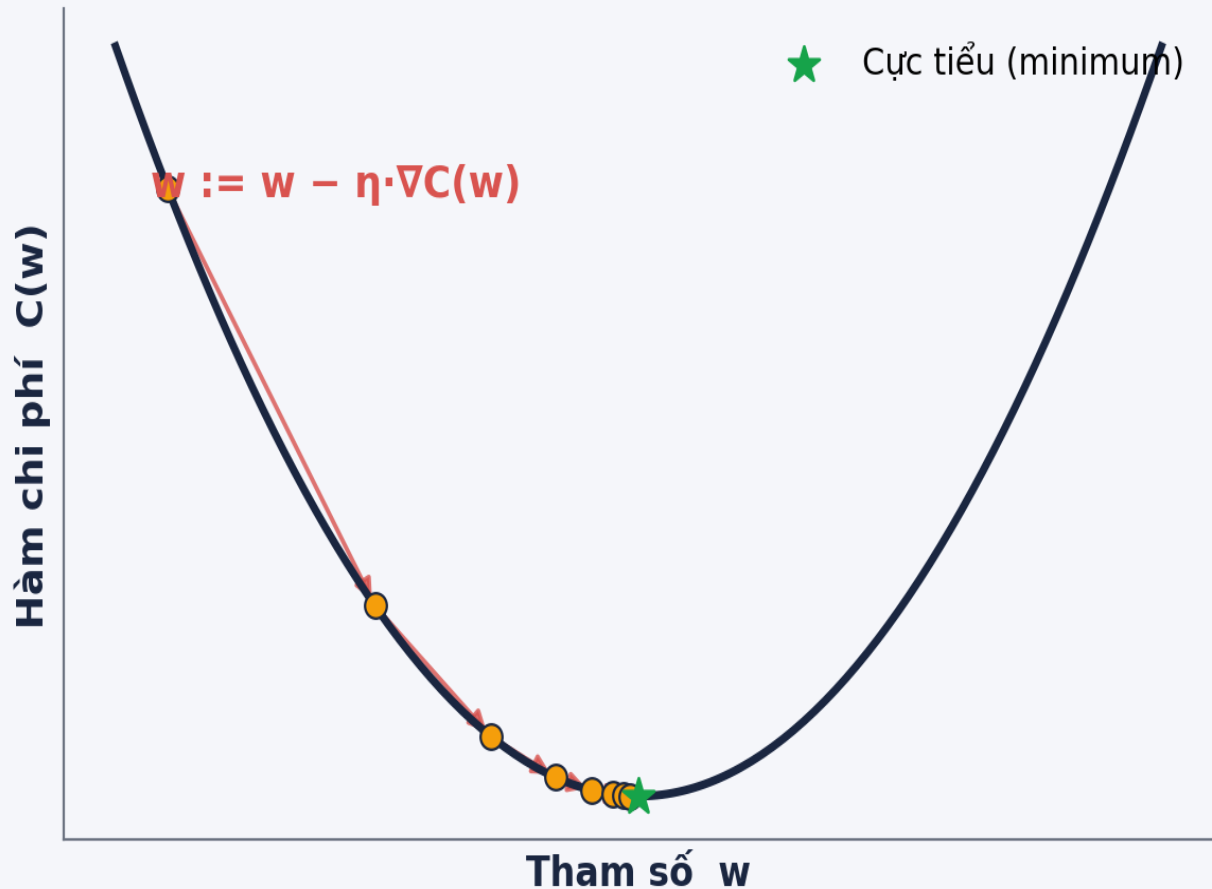
Tiêu chí	Normal Equation	Gradient Descent
Cách tìm nghiệm	Công thức đóng, 1 bước	Lặp dần đến hội tụ
Tốc độ học η	Không cần	Phải chọn/điều chỉnh
Chuẩn hóa đặc trưng	Không bắt buộc	Nên có (hội tụ nhanh)
Số đặc trưng p lớn	Chậm — $O(p^3)$	Tốt — $O(p)$ mỗi bước
$X^T X$ không khả nghịch	Thất bại	Vẫn chạy được
Phù hợp khi	p nhỏ, dữ liệu vừa	p lớn, dữ liệu rất lớn

03

Gradient Descent

Tối ưu hóa bằng hạ gradient

Gradient Descent (Hạ gradient)



- Ý tưởng: đi ngược hướng gradient của hàm chi phí để giảm dần về cực tiểu.

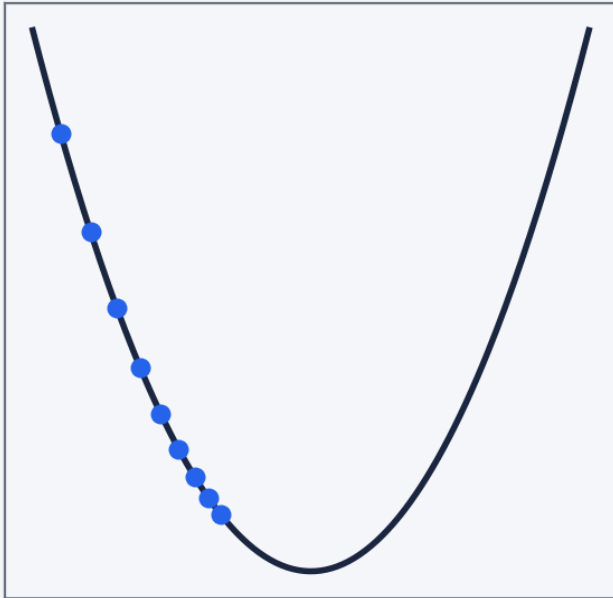
- Quy tắc cập nhật:

$$w := w - \eta \nabla_w C(w)$$

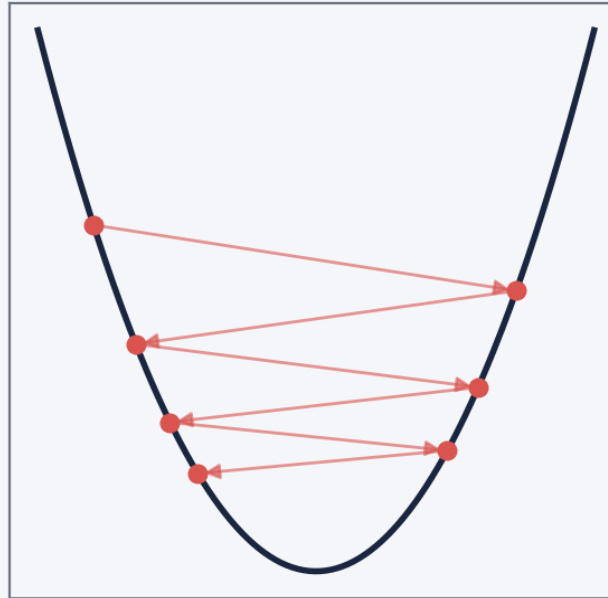
- η (tốc độ học): độ dài mỗi bước.
- $\nabla C(w)$: gradient — hướng dốc nhất của chi phí.
- Lặp đến khi đạt tiêu chí hội tụ (chi phí gần như không giảm nữa).

Tốc độ học & Cực tiểu địa phương

Tốc độ học quá NHỎ
→ hội tụ chậm



Tốc độ học quá LỚN
→ phân kỳ / dao động



Cực tiểu địa phương
vs toàn cục



- **Tốc độ học η quá nhỏ** → hội tụ rất chậm. **Quá lớn** → vượt qua cực tiểu, dao động hoặc phân kỳ.
- **Hàm chi phí không lồi** có thể có nhiều cực tiểu địa phương; gradient descent có thể mắc kẹt (với hồi quy tuyến tính + MSE, chi phí LỖI nên chỉ có 1 cực tiểu toàn cục).

Stochastic GD & Mini-batch

- **Vấn đề:** GD “batch” tính gradient trên TOÀN BỘ N mẫu mỗi bước $\rightarrow$ rất chậm khi N lớn. Hàm chi phí thực chất là kỳ vọng trên phân phối dữ liệu (thường chưa biết).
- **Giải pháp ngẫu nhiên:** ước lượng gradient từ 1 mẫu (SGD) hoặc một lô nhỏ B mẫu (mini-batch).

$$w := w - \eta \nabla_w C_i(w) \quad (1 \text{ mu} / \text{ mini-batch})$$

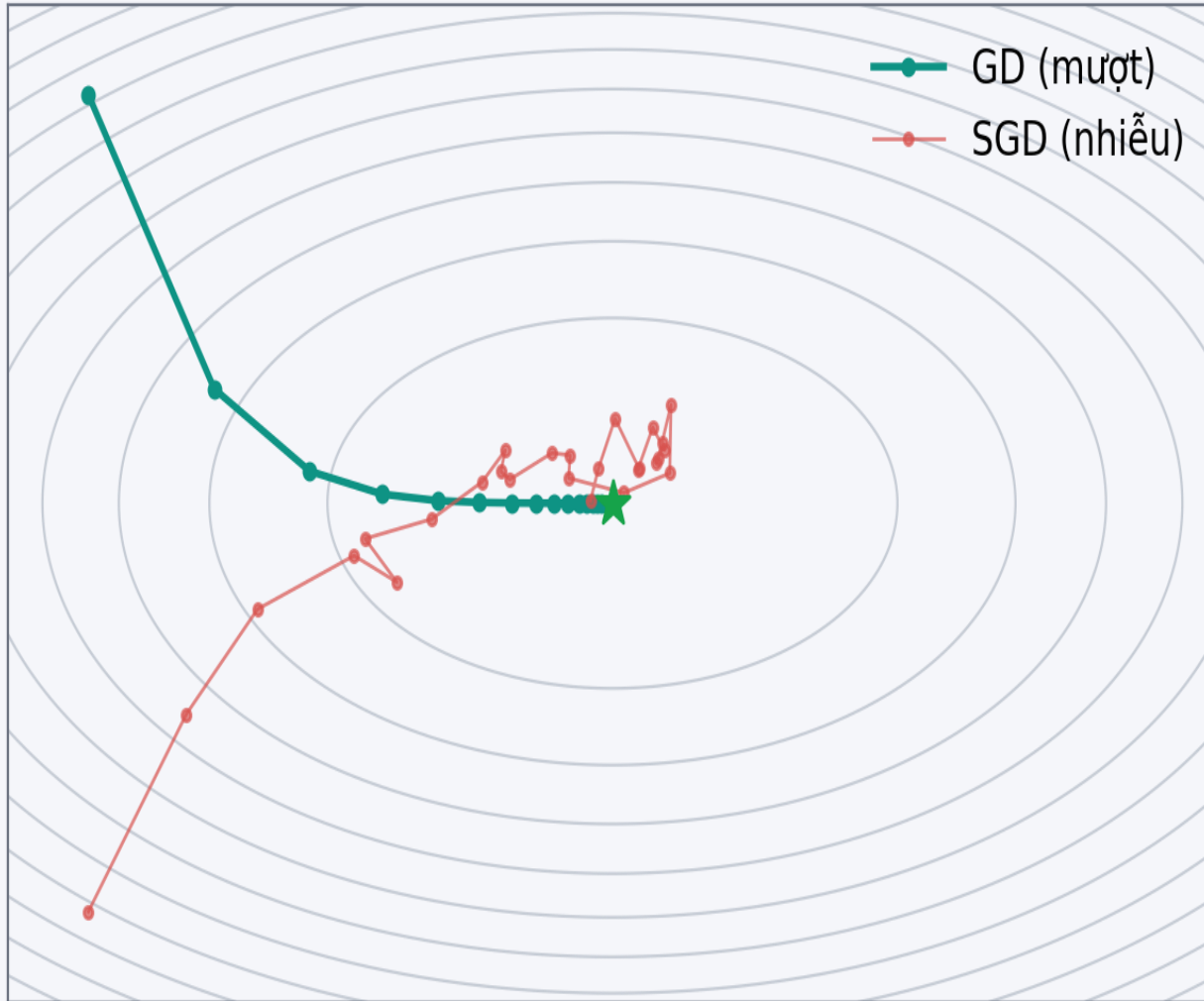
Ưu điểm

- Mỗi bước nhanh hơn nhiều; hỗ trợ học trực tuyến (online); hội tụ nhanh trên dữ liệu lớn.

Mini-batch (32/64/128)

- Dung hòa: ổn định hơn SGD, nhanh hơn batch GD — lựa chọn phổ biến nhất.

SGD vs GD



Batch GD

- Đường đi mượt, ổn định.
- Chậm với dữ liệu lớn; cần toàn bộ dữ liệu trong bộ nhớ.

SGD / Mini-batch

- Nhanh, hỗ trợ online; nhưng gradient nhiễu.
- Dao động quanh cực tiểu; cần điều chỉnh tốc độ học (lịch giảm η).

04

Giả định & Chẩn đoán mô hình

Assumptions & Diagnostics

Các giả định của hồi quy tuyến tính

Tuyến tính

Quan hệ giữa X và Y là tuyến tính (theo tham số).

Độc lập

Các phần dư độc lập với nhau (không tự tương quan).

Phương sai đồng nhất

Phương sai phần dư không đổi theo X (homoscedasticity).

Phần dư chuẩn

Phần dư phân phối xấp xỉ chuẩn (quan trọng cho suy diễn thống kê).

Không đa cộng tuyến

Các đặc trưng không tương quan mạnh với nhau.

Đa cộng tuyến (Multicollinearity)

Ma trận tương quan đặc trưng

Diện tích	1.00	0.93	0.46	-0.21
Số phòng	0.93	1.00	0.41	-0.18
Số tầng	0.46	0.41	1.00	-0.09
Tuổi nhà	-0.21	-0.18	-0.09	1.00

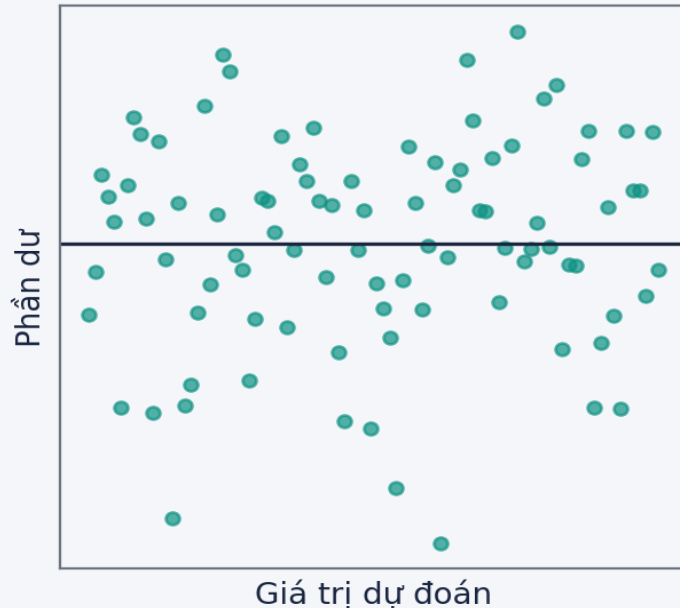
- **Là gì?** hai hay nhiều đặc trưng tương quan **MẠNH** với nhau (vd: Diện tích $\leftrightarrow$ Số phòng = 0.93).
- **Hệ quả:** hệ số β không ổn định, khó diễn giải; sai số chuẩn lớn.
- **Phát hiện:**
 - Ma trận tương quan; chỉ số VIF.

$$VIF_j = \frac{1}{1 - R_j^2}$$

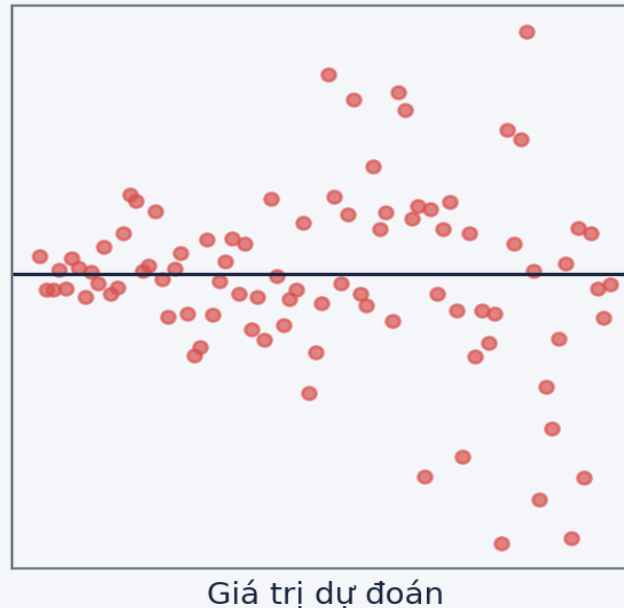
VIF > 5–10 $\rightarrow$ đa cộng tuyến đáng lo.
Khắc phục: bỏ bớt đặc trưng, gộp, hoặc dùng Ridge/PCA.

Phân tích phần dư (Residual Analysis)

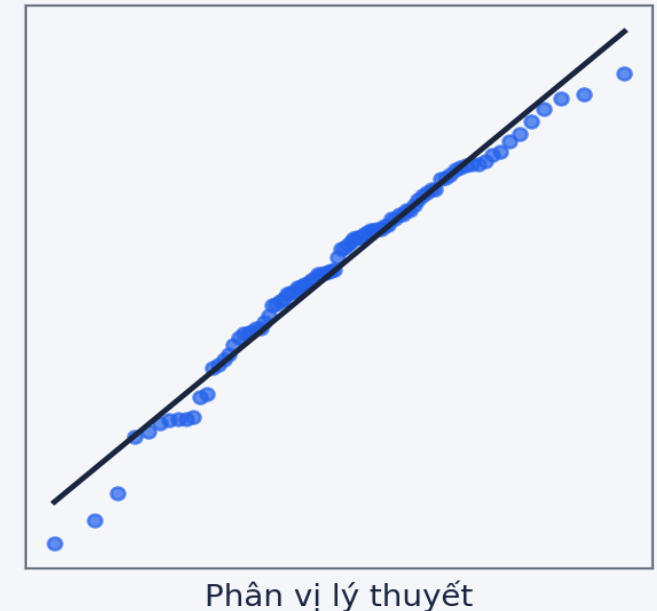
Tốt: phần dư phân tán ngẫu nhiên



Xấu: hình phễu → phương sai không đồng nhất



Q-Q plot: phần dư có phân phối chuẩn?



- **Phần dư–giá trị dự đoán:** phân tán ngẫu nhiên quanh 0 → mô hình tốt; có hình phễu → phương sai không đồng nhất.
- **Q-Q plot:** điểm bám sát đường chéo → phần dư phân phối chuẩn. Lệch nhiều → vi phạm giả định, cần biến đổi dữ liệu hoặc đổi mô hình.

05

Đánh giá mô hình

Model Evaluation

RSE & R² (Hệ số xác định)

$$RSE = \sqrt{\frac{1}{n-2} RSS}$$

$$R^2 = 1 - \frac{RSS}{TSS}, \quad TSS = \sum (y_i - \bar{y})^2$$

- **RSE:** sai số chuẩn của phần dư — độ lệch trung bình của dự đoán so với thực tế (cùng đơn vị Y).
- **R²:** tỷ lệ phương sai của Y được mô hình giải thích. 0 = không giải thích gì; 1 = khớp hoàn hảo.
- **Ví dụ:** R² = 0.85 nghĩa là mô hình giải thích 85% biến thiên của Y.

Bộ đo đo hồi quy đầy đủ

MAE

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Sai số tuyệt đối trung bình — cùng đơn vị Y, ít nhạy ngoại lai.

MSE

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Bình phương sai số trung bình — phạt nặng sai số lớn (hàm mất mát).

RMSE

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Căn của MSE — cùng đơn vị Y, dễ diễn giải nhất.

Adjusted R²

$$R_{adj}^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

R² hiệu chỉnh theo số đặc trưng p — phạt khi thêm biến vô ích.

R² & Adjusted R²: càng gần 1 càng tốt · MAE/MSE/RMSE: càng nhỏ càng tốt

06

Thực hành với Python

Hands-on with Python

Quy trình xây dựng & đánh giá mô hình



Nguyên tắc vàng: chỉ đánh giá trên dữ liệu mô hình CHƯA thấy (tập test) để ước lượng hiệu năng thực tế.

Cài đặt thuần NumPy

```
import numpy as np

# Thêm cột bias (x0 = 1) vào ma trận đặc trưng
X_b = np.c_[np.ones((m, 1)), X]

# --- Cách 1: Normal Equation (nghiệm dạng đóng) ---
beta = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y

# --- Cách 2: Gradient Descent ---
beta = np.zeros((n + 1, 1))
eta, epochs = 0.01, 1000
for _ in range(epochs):
    grad = (2/m) * X_b.T @ (X_b @ beta - y) # đạo hàm MSE
    beta -= eta * grad

y_pred = X_b @ beta # dự đoán
```

Dùng scikit-learn: huấn luyện & đánh giá

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

reg = LinearRegression().fit(X_tr, y_tr)    # huấn luyện
y_pred = reg.predict(X_te)                 # dự đoán

# Đánh giá
rmse = np.sqrt(mean_squared_error(y_te, y_pred))
r2    = r2_score(y_te, y_pred)

# Phân tích phần dư
res = y_te - y_pred
plt.scatter(y_pred, res); plt.axhline(0)
```

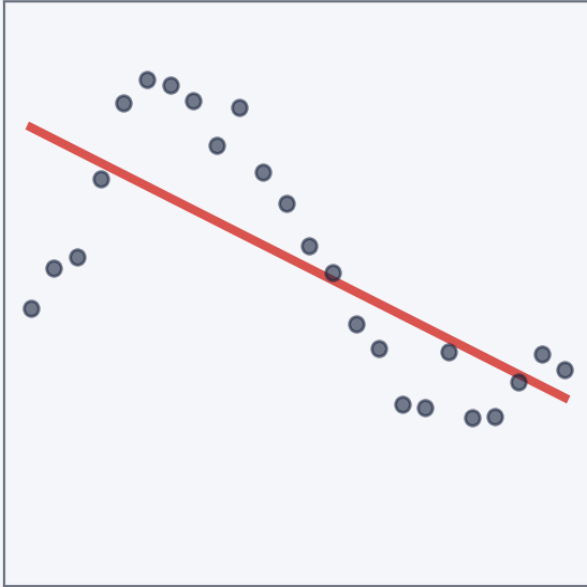
07

Mở rộng: Hồi quy phi tuyến

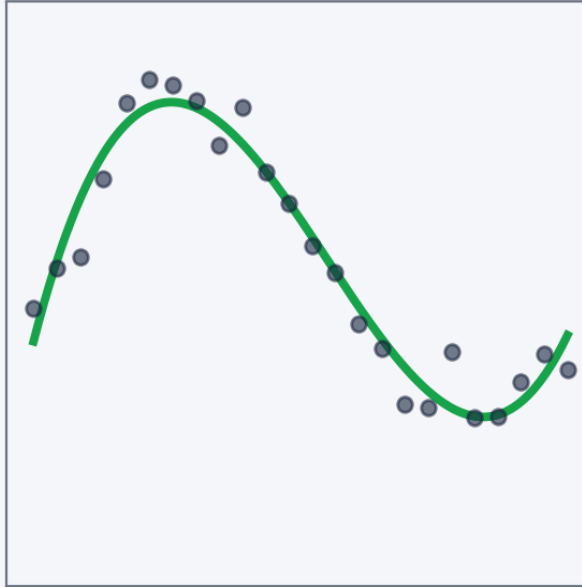
Beyond Linear: Polynomial Regression

Hồi quy đa thức (Polynomial Regression)

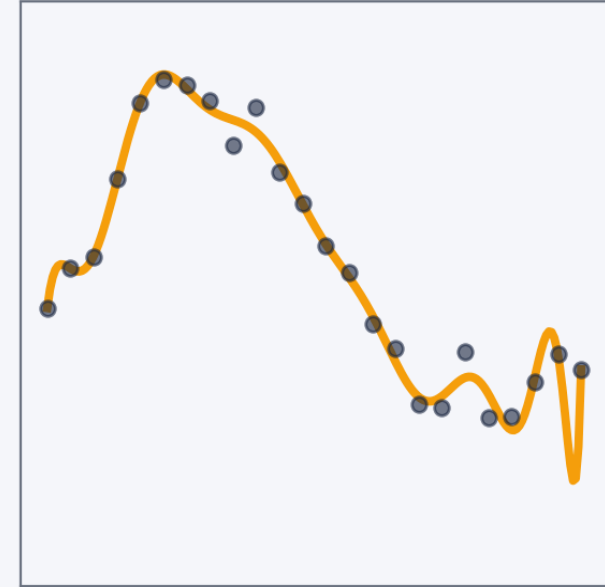
Bậc 1 — dưới khớp



Bậc 3 — khớp tốt



Bậc 15 — quá khớp



$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \dots + \beta_d X^d$$

- Khi quan hệ X–Y KHÔNG tuyến tính, khớp đa thức bậc cao hơn. Ứng dụng: đường cong tăng trưởng, lợi ích giảm dần của chi quảng cáo.

⚠ Cảnh báo: bậc càng cao càng dễ QUÁ KHỚP → dùng kiểm định chéo để chọn bậc.

Tăng cường đặc trưng: PolynomialFeatures

Mẹo: “tăng cường” đặc trưng ($X \rightarrow 1, X, X^2 \dots$) biến bài toán phi tuyến thành tuyến tính theo tham số $\rightarrow$ vẫn dùng được LinearRegression.

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression

# degree=2:  X  ->  [1, X, X^2]
model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
model.fit(X_train, y_train)
model.score(X_test, y_test)
```